

KALIMAT CASE OF dan While do loop

ISA212 Basisdata 2

minggu 5

Mira Musrini Barmawi M.T.

DEFINITION

- The case statement in SQL returns a value on a specified condition. We can use a Case statement in select queries along with Where, Order By and Group By clause. It can be used in Insert statement as well. In this article, we would explore the CASE statement and its various use cases.
- Suppose you have a table that stores the ProductID for all products in a mini-store. You want to get Productname for a particular ProductID.

example

- Look at the following example; We declared a variable @ProductID and specified value 1 for it. In Case statement, we defined conditions. Once a condition is satisfied, its corresponding value is returned.

```
DECLARE @ProductID int
SET @ProductID = 1
SELECT
    Case @ProductID
    WHEN 1 THEN 'Bread and Biscuits'
    WHEN 2 THEN 'Confectioneries'
    WHEN 3 THEN 'Fruits and Vergetables'
    ELSE 'No such product'
    END
```

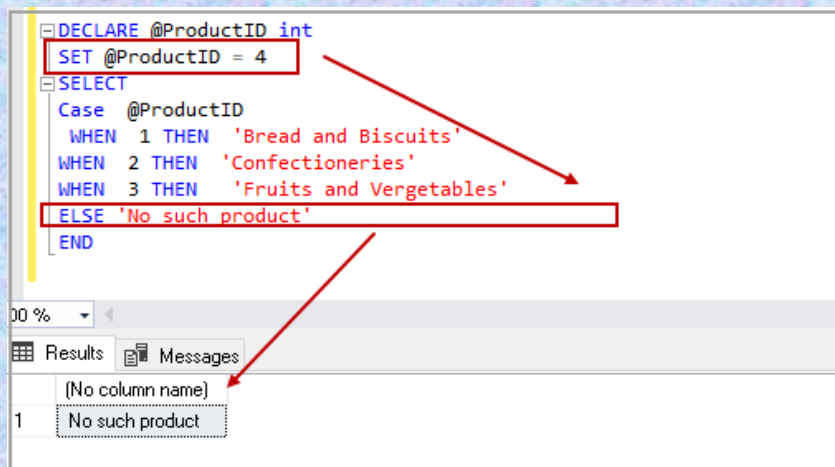
100 %

Results Messages

	(No column name)
1	Bread and Biscuits

example

Similarly, if we change the condition in a Case statement in SQL, it returns appropriate expression. In the following example, we want to get Product name for ProductID 4. It does not satisfy Case statement condition; therefore, it gave output from Else expression.



```
DECLARE @ProductID int
SET @ProductID = 4
SELECT
  Case @ProductID
    WHEN 1 THEN 'Bread and Biscuits'
    WHEN 2 THEN 'Confectioneries'
    WHEN 3 THEN 'Fruits and Vegetables'
    ELSE 'No such product'
  END
```

	(No column name)
1	No such product

Exercise

- Let us explore a few examples of the Case statement in SQL. Before we proceed, create a sample table and insert few records in it.

exercise

```
USE [SQLShackDemo]
GO
CREATE TABLE dbo.Employee
(
  EmployeeID INT IDENTITY PRIMARY KEY,
  EmployeeName VARCHAR(100) NOT NULL,
  Gender VARCHAR(1) NOT NULL,
  StateCode VARCHAR(20) NOT NULL,
  Salary money NOT NULL,
)
GO
USE [SQLShackDemo]
GO
```

```
SET IDENTITY_INSERT [dbo].[Employee] ON
```

```
GO
```

```
INSERT [dbo].[Employee] ([EmployeeID], [EmployeeName], [Gender],  
[StateCode], [Salary]) VALUES (201, N'Jerome', N'M', N'FL', 83000.0000)
```

```
GO
```

```
INSERT [dbo].[Employee] ([EmployeeID], [EmployeeName], [Gender],  
[StateCode], [Salary]) VALUES (202, N'Ray', N'M', N'AL', 88000.0000)
```

```
GO
```

```
INSERT [dbo].[Employee] ([EmployeeID], [EmployeeName], [Gender],  
[StateCode], [Salary]) VALUES (203, N'Stella', N'F', N'AL', 76000.0000)
```

```
GO
```

```
INSERT [dbo].[Employee] ([EmployeeID], [EmployeeName], [Gender],  
[StateCode], [Salary]) VALUES (204, N'Gilbert', N'M', N'Ar', 42000.0000)
```

```
GO
```

```
INSERT [dbo].[Employee] ([EmployeeID], [EmployeeName], [Gender],  
[StateCode], [Salary]) VALUES (205, N'Edward', N'M', N'FL', 93000.0000)
```

```
GO
```

```
INSERT [dbo].[Employee] ([EmployeeID], [EmployeeName], [Gender],  
[StateCode], [Salary]) VALUES (206, N'Ernest', N'F', N'Al', 64000.0000)
```

```
GO
```

```
INSERT [dbo].[Employee] ([EmployeeID], [EmployeeName], [Gender],  
[StateCode], [Salary]) VALUES (207, N'Jorge', N'F', N'IN', 75000.0000)
```

```
GO
```

```
INSERT [dbo].[Employee] ([EmployeeID], [EmployeeName], [Gender],  
[StateCode], [Salary]) VALUES (208, N'Nicholas', N'F', N'Ge', 71000.0000)
```

```
GO
```

```
INSERT [dbo].[Employee] ([EmployeeID], [EmployeeName], [Gender],  
[StateCode], [Salary]) VALUES (209, N'Lawrence', N'M', N'IN', 95000.0000)
```

```
GO
```

```
INSERT [dbo].[Employee] ([EmployeeID], [EmployeeName], [Gender],  
[StateCode], [Salary]) VALUES (210, N'Salvador', N'M', N'Co', 75000.0000)
```

```
GO
```

```
SET IDENTITY_INSERT [dbo].[Employee] OFF
```

```
GO
```

We have following records in Employee table.

Results		Messages				
	EmployeeID	EmployeeName	Gender	StateCode	Salary	
1	201	Jerome	M	FL	83000.00	
2	202	Ray	M	AL	88000.00	
3	203	Stella	F	AL	76000.00	
4	204	Gilbert	M	Ar	42000.00	
5	205	Edward	M	FL	93000.00	
6	206	Ernest	F	Al	64000.00	
7	207	Jorge	F	IN	75000.00	
8	208	Nicholas	F	Ge	71000.00	
9	209	Lawrence	M	IN	95000.00	
10	210	Salvador	M	Co	75000.00	

A simple CASE statement expression

In this format, we evaluate one expression against multiple values. In a simple case statement, it evaluates conditions one by one. Once the condition and expression are matched, it returns the expression mentioned in THEN clause.

We have following syntax for a case statement in SQL with a simple expression

A simple CASE statement expression

```
SELECT CASE Expression  
When expression1 Then Result1  
When expression2 Then Result2  
...  
ELSE Result  
END
```

A simple CASE statement expression

Usually, we store abbreviations in a table instead of its full form. For example, in my Employee table, I have used abbreviations in Gender and StateCode. I want to use a Case statement to return values as Male and Female in the output instead of M and F.

Execute the following code and notice that we want to evaluate CASE Gender in this query.

A simple CASE statement expression

```
Select EmployeeName,  
CASE  
WHEN gender = 'F' THEN 'FEMALE'  
WHEN gender = 'M' THEN 'MALE'  
Else 'Unknown Gender'  
END AS gender  
from Employee
```

A simple CASE statement expression

```
Select EmployeeName,  
       CASE  
WHEN Salary >=80000 AND Salary <=100000 THEN 'Director'  
WHEN Salary >=50000 AND Salary <80000 THEN 'Senior  
Consultant'  
Else 'Director'  
END AS Designation  
from Employee
```

CASE STATEMENT

- SQL Server CASE statement is equivalent to the IF-THEN statement in Excel.
- The CASE statement is used to implement the logic where you want to set the value of one column depending upon the values in other columns.
- The SQL Server CASE Statement consists of at least one pair of WHEN and THEN statements. The WHEN statement specifies the condition to be tested. The THEN statement specifies the action if the WHEN condition returns TRUE.

CASE STATEMENT

- The ELSE statement is optional and executes when none of the WHEN conditions return true. The CASE statement ends with an END keyword.
- In this article, we will take a look at a number of different examples of the CASE statement. But before we do that, we'll create some dummy data to work with.

Creating dummy data

```
CREATE Database ShowRoom;
```

```
GO
```

```
USE ShowRoom;
```

```
CREATE TABLE Cars
```

```
(
```

```
    id INT,
```

```
    name VARCHAR(50) NOT NULL,
```

```
    company VARCHAR(50) NOT NULL,
```

```
    power INT NOT NULL,
```

```
    color VARCHAR(50) NOT NULL,
```

```
    model INT NOT NULL,
```

```
    condition VARCHAR(50) NOT NULL
```

```
)
```

Creating dummy data

- Now let's insert some dummy data into the Cars table. Execute the following script:

USE ShowRoom

```
INSERT INTO Cars VALUES
(1, 'Corrolla', 'Toyota', 1800, 'red', 1995, 'X'),
(2, 'City', 'Honda', 1500 , 'black', 2015, 'X'),
(3, 'C200', 'Mercedez', 2000 , 'white', 1992, 'X'),
(4, 'Vitz', 'Toyota', 1300 , 'blue', 2007, 'X'),
(5, 'Baleno', 'Suzuki', 1500 , 'white', 2012, 'X'),
(6, 'C500', 'Mercedez', 5000 , 'grey', 1994, 'X'),
(7, '800', 'BMW', 8000 , 'blue', 2016, 'X'),
(8, 'Mustang', 'Ford', 5000 , 'red', 1997, 'X'),
(9, '208', 'Peugeot', 5400, 'black', 1999, 'X'),
(10, 'Prius', 'Toyota', 3200 , 'red', 2003, 'X')
```

CARS TABLE

```
SELECT * FROM Cars
```

 Results  Messages							
	id	name	company	power	color	model	condition
1	1	Corrolla	Toyota	1800	red	1995	X
2	2	City	Honda	1500	black	2015	X
3	3	C200	Mercedez	2000	white	1992	X
4	4	Vitz	Toyota	1300	blue	2007	X
5	5	Baleno	Suzuki	1500	white	2012	X
6	6	C500	Mercedez	5000	grey	1994	X
7	7	800	BMW	8000	blue	2016	X
8	8	Mustang	Ford	5000	red	1997	X
9	9	208	Peugeot	5400	black	1999	X
10	10	Prius	Toyota	3200	red	2003	X

SQL Server CASE statement syntax

The syntax of the CASE statement is pretty straight forward:

```
SELECT column1,  
        column2,  
        CASE WHEN CONDITION THEN 'Value1'  
        ELSE 'Value2' END AS columnX  
FROM Cars
```

The CASE statement has to be included inside the SELECT Statement. It starts with the CASE keyword followed by the WHEN keyword and then the CONDITION.

CASE statement examples

Let's now see the CASE statement in action.

```
SELECT name,  
       model,  
       CASE WHEN model > 2000 THEN 'New'  
            ELSE 'Old' END AS condition  
FROM Cars
```

Results				Messages			
	name	model	condition				
1	Corolla	1995	Old				
2	City	2015	New				
3	C200	1992	Old				
4	Vitz	2007	New				
5	Baleno	2012	New				
6	C500	1994	Old				
7	800	2016	New				
8	Mustang	1997	Old				
9	208	1999	Old				
10	Prius	2003	New				

Multiple conditions in CASE statement

Look at the following script:

```
SELECT name,  
       model,  
       CASE WHEN model > 2010 THEN 'New'  
       WHEN model > 2000 THEN 'Average'  
       WHEN model > 1990 THEN 'Old'  
       ELSE 'Old' END AS condition  
FROM Cars
```

Multiple conditions in CASE statement

Results		Messages	
	name	model	condition
1	Corrolla	1995	Old
2	City	2015	New
3	C200	1992	Old
4	Vitz	2007	Average
5	Baleno	2012	New
6	C500	1994	Old
7	800	2016	New
8	Mustang	1997	Old
9	208	1999	Old
10	Prius	2003	Average

Multiple conditions in CASE statement

However, in the above script, the conditions are overlapping as. the model with a value greater than 2010 also has a value greater than 2000 and 1990. A better way to implement multiple conditions is to use logical operators like AND, OR, NOT, etc. Look at the following script:

```
SELECT name,  
       model,  
       CASE WHEN model > 2010 THEN 'New'  
            WHEN model > 2000 AND model <2010 THEN 'Average'  
            WHEN model > 1990 AND model <2000 THEN 'Old'  
            ELSE 'Old' END AS condition  
FROM Cars
```

Multiple conditions in CASE statement

The output of the script above looks like this:

Results				Messages			
	name	model	condition				
1	Corrolla	1995	Old				
2	City	2015	New				
3	C200	1992	Old				
4	Vitz	2007	Average				
5	Baleno	2012	New				
6	C500	1994	Old				
7	800	2016	New				
8	Mustang	1997	Old				
9	208	1999	Old				
10	Prius	2003	Average				

Multiple conditions in CASE statement

```
SELECT name,  
       color,  
       model,  
       CASE WHEN model > 2010 AND color = 'white' THEN 'New White'  
       WHEN model > 2010 THEN 'New'  
       WHEN model > 2000 AND model <2010 THEN 'Average'  
       WHEN model > 1990 AND model <2000 THEN 'Old'  
       ELSE 'Old' END AS condition  
FROM Cars
```

Multiple conditions in CASE statement

Results		Messages		
	name	color	model	condition
1	Corrolla	red	1995	Old
2	City	black	2015	New
3	C200	white	1992	Old
4	Vitz	blue	2007	Average
5	Baleno	white	2012	New White
6	C500	grey	1994	Old
7	800	blue	2016	New
8	Mustang	red	1997	Old
9	208	black	1999	Old
10	Prius	red	2003	Average

Using GROUP BY with SQL Server CASE statement

The CASE statement can also be used in conjunction with the GROUP BY statement in order to apply aggregate functions.

For example, if we want to count the number of new (model number greater than 2000) and old (model number less than 2000) vehicles, we can use the GROUP BY clause with the CASE statement as follows:

Using GROUP BY with SQL Server CASE statement

```
SELECT
    CASE WHEN model > 2000 THEN 'New'
         ELSE 'Old' END AS condition,
    COUNT(1) AS count
FROM Cars
GROUP BY CASE WHEN model > 2000 THEN 'New'
         ELSE 'Old' END
```

Using GROUP BY with SQL Server CASE statement

Results		Messages
	condition	count
1	New	5
2	Old	5

Using GROUP BY with SQL Server CASE statement

Similarly, we can GROUP BY more than two values. Look at the following script:

```
SELECT
    CASE WHEN model > 2010 THEN 'New'
    WHEN model > 2000 THEN 'Average'
    WHEN model > 1990 THEN 'Old'
    ELSE 'Old' END AS condition,
    COUNT(1) AS count
FROM Cars
GROUP BY CASE WHEN model > 2010 THEN 'New'
    WHEN model > 2000 THEN 'Average'
    WHEN model > 1990 THEN 'Old'
    ELSE 'Old' END
```

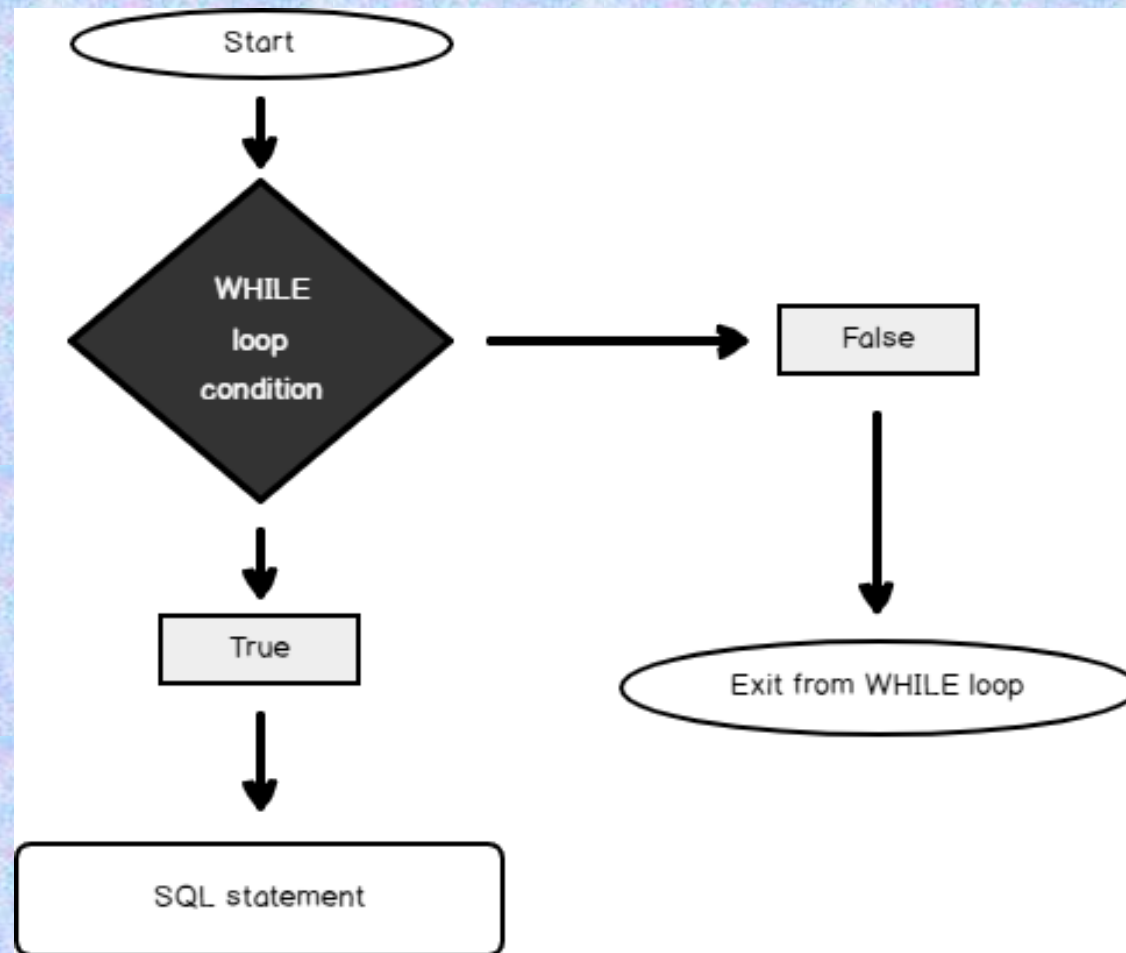
Using GROUP BY with SQL Server CASE statement

Results		Messages
	condition	count
1	Average	2
2	New	3
3	Old	5

WHILE LOOP STATEMENT

SQL WHILE loop provides us with the advantage to execute the SQL statement(s) repeatedly until the specified condition result turn out to be false.

In the following sections of this article, we will use more flowcharts in order to explain the notions and examples. For this reason, firstly, we will explain what is a flowchart briefly. The flowchart is a visual geometric symbol that helps to explain algorithms visually. The flowchart is used to simply design and document the algorithms. In the flowchart, each geometric symbol specifies different meanings.



SQL WHILE loop syntax and example

The syntax of the WHILE loop in SQL looks like as follows:

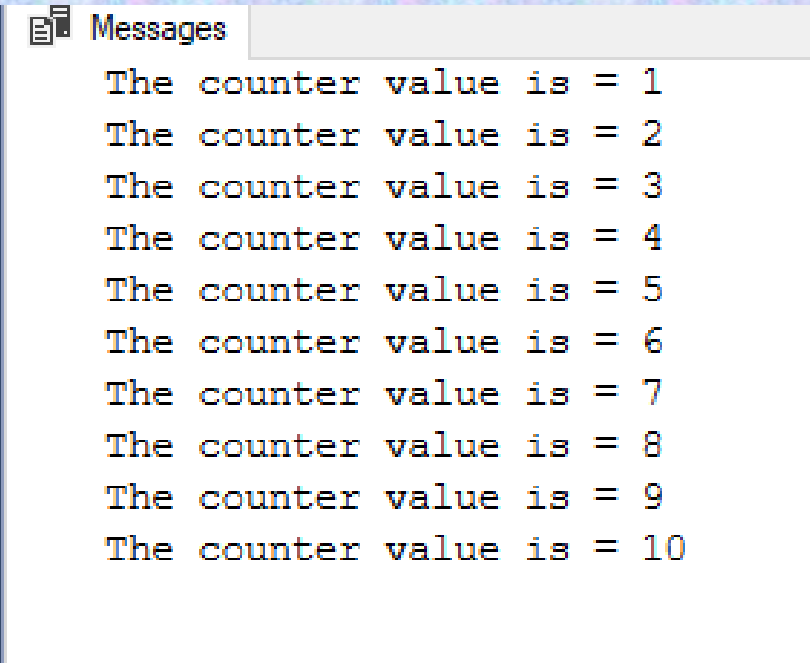
```
WHILE condition
BEGIN
    {...statements...}
END
```

example

- After these explanations, we will give a very simple example of a WHILE loop in SQL. In the example given below, the WHILE loop example will write a value of the variable ten times, and then the loop will be completed:

```
DECLARE @Counter INT
SET @Counter=1
WHILE ( @Counter <= 10)
BEGIN
    PRINT 'The counter value is = ' + CONVERT(VARCHAR,@Counter)
    SET @Counter = @Counter + 1
END
```

result



The Messages window displays a series of ten log messages. Each message is a single line of text in a monospaced font, stating 'The counter value is = ' followed by a number from 1 to 10. The messages are stacked vertically, with each new message appearing below the previous one. The window has a title bar that says 'Messages' and a small icon on the left.

```
The counter value is = 1  
The counter value is = 2  
The counter value is = 3  
The counter value is = 4  
The counter value is = 5  
The counter value is = 6  
The counter value is = 7  
The counter value is = 8  
The counter value is = 9  
The counter value is = 10
```

- In this part of the code, we declare a variable, and we assign an initializing value to it:

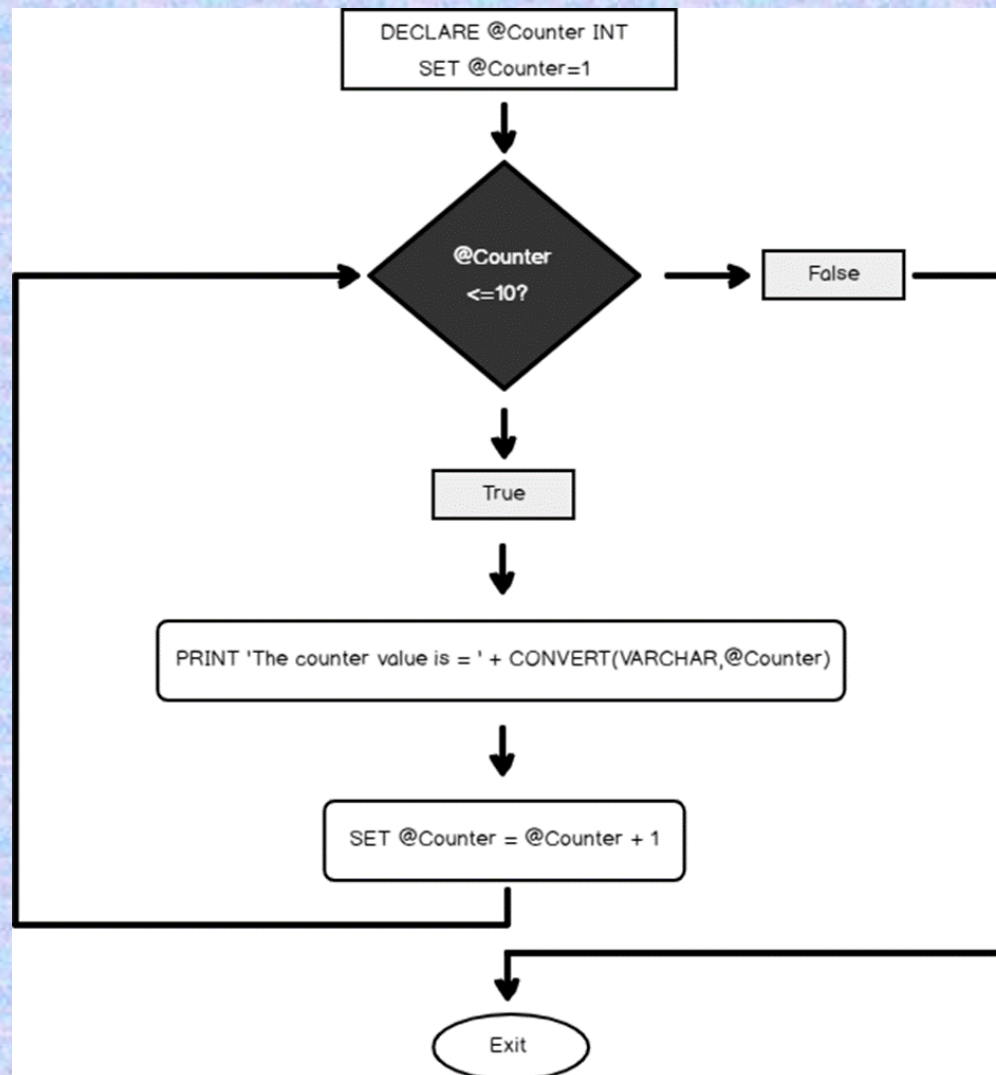
```
DECLARE @Counter INT  
SET @Counter=1
```

- This part of the code has a specified condition that until the variable value reaches till 10, the loop continues and executes the PRINT statement. Otherwise, the while condition will not occur, and the loop will end:

```
WHILE ( @Counter <= 10)
```

In this last part of the code, we executed the SQL statement, and then we incremented the value of the variable:

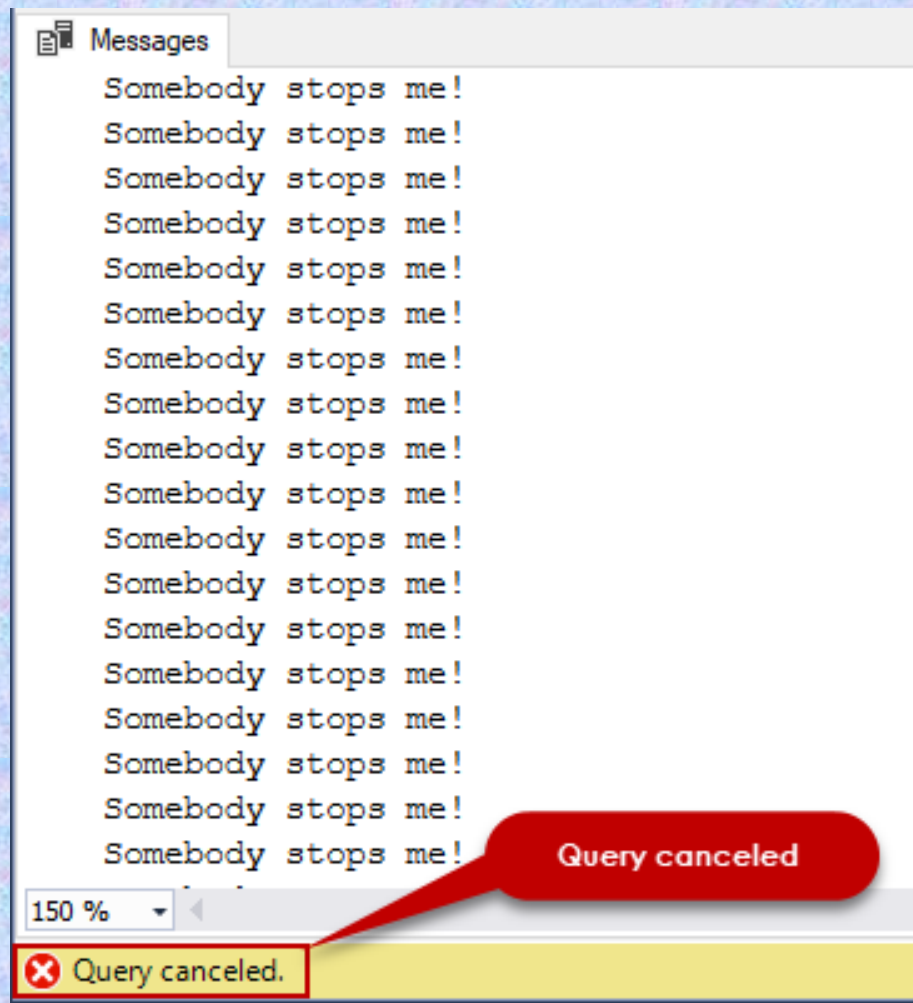
The following flowchart illustrates this WHILE loop example visually:

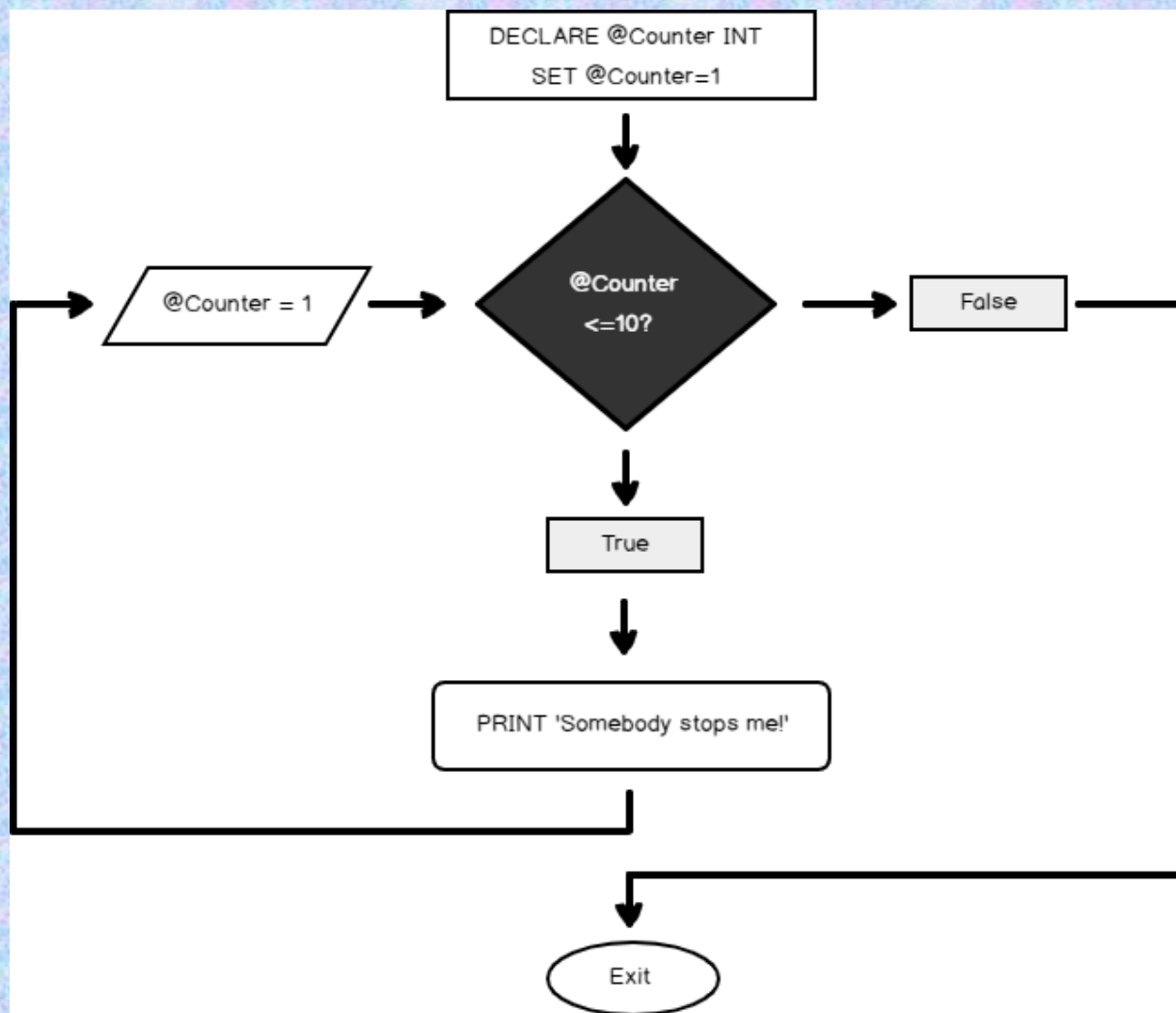


Infinite SQL WHILE loop

In the infinite loop AKA endless loop, the condition result will never be false, so the loop never ends and can work forever. Imagine that we have a WHILE loop, and we don't increment the value of the variable. In this scenario, the loop runs endlessly and never ends. Now, we will realize this scenario with the help of the following example. We need to take account of one thing that we should not forget to cancel the execution of the query manually:

```
DECLARE @Counter INT
SET @Counter=1
WHILE ( @Counter <= 10)
BEGIN
    PRINT 'Somebody stops me!'
END
```





BREAK STATEMENT

BREAK statement is used in the SQL WHILE loop in order to exit the current iteration of the loop immediately when certain conditions occur. In the generally IF...ELSE statement is used to check whether the condition has occurred or not. Refer to the SQL IF Statement introduction and overview article for more details about the IF...ELSE statement.

The following example shows the usage of the BREAK statement in the WHILE loop:

BREAK STATEMENT

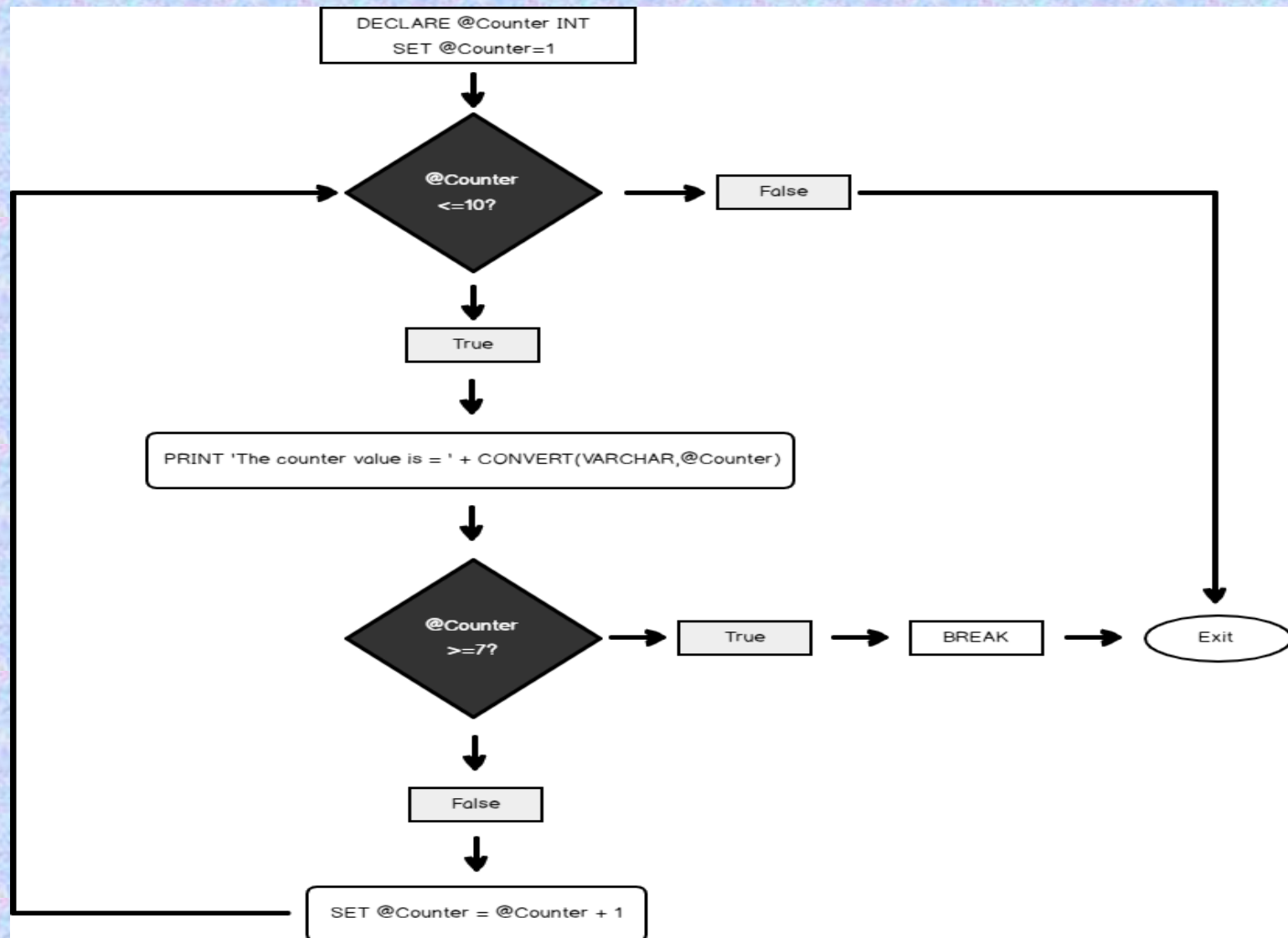
```
DECLARE @Counter INT
SET @Counter=1
WHILE ( @Counter <= 10)
BEGIN
    PRINT 'The counter value is = ' + CONVERT(VARCHAR,@Counter)
    IF @Counter >=7
    BEGIN
        BREAK
    END
    SET @Counter  = @Counter  + 1
END
```

BREAK STATEMENT



Messages

```
The counter value is = 1  
The counter value is = 2  
The counter value is = 3  
The counter value is = 4  
The counter value is = 5  
The counter value is = 6  
The counter value is = 7
```



CONTINUE statement

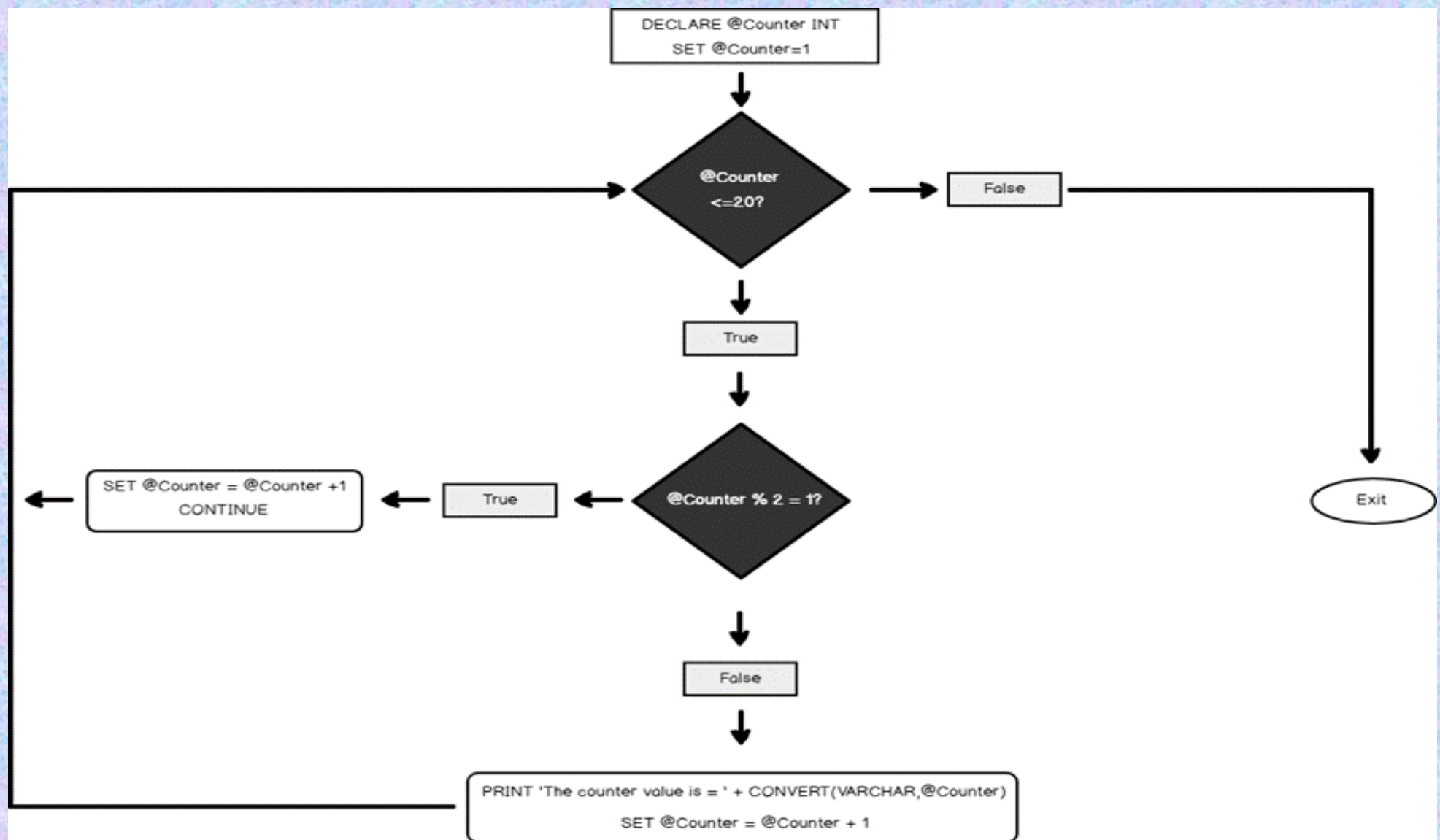
In the following example, we will check whether the variable value is odd or even. If the variable value is odd, the code enters the IF...ELSE statement blocks and increment the value of the variable, execute the CONTINUE statement and starts a new iteration:

```
DECLARE @Counter INT
SET @Counter=1
WHILE ( @Counter <= 20)
BEGIN
    IF @Counter % 2 =1
    BEGIN
        SET @Counter  = @Counter  + 1
        CONTINUE
    END
    PRINT 'The counter value is = ' + CONVERT(VARCHAR,@Counter)
    SET @Counter  = @Counter  + 1
END
```

CONTINUE statement

Messages

```
The counter value is = 2  
The counter value is = 4  
The counter value is = 6  
The counter value is = 8  
The counter value is = 10  
The counter value is = 12  
The counter value is = 14  
The counter value is = 16  
The counter value is = 18  
The counter value is = 20
```



Reading table records through the WHILE loop

In the following example, we will read table data, row by row. Firstly we will create a sample table:

```
USE tempdb
GO
DROP TABLE IF EXISTS SampleTable
CREATE TABLE SampleTable
(Id INT, CountryName NVARCHAR(100), ReadStatus TINYINT)
GO
INSERT INTO SampleTable ( Id, CountryName, ReadStatus)
Values  (1, 'Germany', 0),
        (2, 'France', 0),
        (3, 'Italy', 0),
        (4, 'Netherlands', 0) ,
        (5, 'Poland', 0)

SELECT * FROM SampleTable
```

Results		Messages	
	Id	CountryName	ReadStatus
1	1	Germany	0
2	2	France	0
3	3	Italy	0
4	4	Netherlands	0
5	5	Poland	0

```
USE tempdb
```

```
GO
```

```
DECLARE @Counter INT , @MaxId INT,
        @CountryName NVARCHAR(100)
SELECT @Counter = min(Id) , @MaxId =
max(Id)
FROM SampleTable
```

```
WHILE(@Counter IS NOT NULL
      AND @Counter <= @MaxId)
BEGIN
    SELECT @CountryName = CountryName
    FROM SampleTable WHERE Id = @Counter

    PRINT CONVERT(VARCHAR,@Counter) + '.
country name is ' + @CountryName
    SET @Counter = @Counter + 1
END
```

RESULT



Messages

1. country name is Germany
2. country name is France
3. country name is Italy
4. country name is Netherlands
5. country name is Poland